

מחברת 4: עיבוד שפה טבעית (NLP) ומידול נושאים

מבוא למדעי הרוח הדיגיטליים | אוניברסיטת אריאל | תשפ"ו

פרופ' שי גורדין

NLP (Natural Language Processing) = עיבוד שפה טבעית – ענף של AI שעוסק בהבנה וניתוח של שפה אנושית.

נושאים: (1) מהו NLP ומה הכלים העיקריים (2) זיהוי ישויות בשם (NER) לעברית (3) ניתוח רגש (Sentiment Analysis) (4) מידול נושאים – LDA (5) ומרחב וקטורי (6) יישומים במדעי הרוח

🎯 **הקשר למחקר:** ניתוח הגניזה הקהירית, קורפוסים של כתבות עיתון, זיהוי דמויות היסטוריות בטקסטים.

חלק א: מהו NLP?

NLP = ממשק בין בלשנות, מדעי המחשב ובינה מלאכותית.

רמות הניתוח הבלשני:

רמה	שם	דוגמה
תו	Character	א, ב, ג
מורפמה	Morpheme	כתב+תי, שמ+ים
מילה	Token	"ירושלים"
משפט	Sentence	"ירושלים היא בירת ישראל"
שיח	Discourse	פסקה, מסמך

מטלות NLP עיקריות:

- Tokenization** – פיצול לטוקנים (מילים/תווים)
- POS Tagging** – תיוג חלקי דיבור (שם עצם, פועל...)
- NER** – זיהוי ישויות בשם (אנשים, מקומות, ארגונים)
- Sentiment Analysis** – ניתוח רגש (חיובי/שלילי/ניטרלי)
- Topic Modeling** – גילוי נושאים נסתרים בקורפוס

NLP לעברית – אתגרים:

- כתיב חסר (ניקוד)
- מורפולוגיה עשירה (שורשים, בניינים)
- אותיות סופיות (מ/ם, נ/ן, פ/ף...)
- כיוון RTL

לקריאה: Goldberg, Y. (2017). *Neural Network Methods for Natural Language Processing*. Morgan & Claypool

לפני שמתחילים – מדריך הגדרה מהירה 🚀

שלב 1: פתחו את המחברת ב-Google Colab

לחצו על הכפתור הכחול "Open in Colab" בדף הקורס.

שלב 2: שמרו עותק אישי

File → Save a copy in Drive
(חובה – אחרת השינויים שלכם לא יישמרו!)

שלב 3: הריצו את התאים לפי הסדר – מלמעלה למטה

- **Shift + Enter** = מריץ תא נוכחי ועובר לבא אחריו
- **Ctrl + Enter** = מריץ תא נוכחי ונשאר בו

זמן משוער ⌚

- שיעור: 75~ דקות
- ⚠️ שימו לב: אימון מודל נושאים (LDA) לוקח 2-5 דקות – זה תקין!

מה נלמד היום 💡

- ניקוי טקסט (Tokenization, Stop Words, Lemmatization)
- מידול נושאים עם LDA
- פרשנות: איך מזהים נושאים בתוצאות?

? נתקעתם?

1. קראו את הודעת השגיאה
2. שאלו את Gemini ב-AI Studio – העתיקו את השגיאה ושאלו
3. שאלו את המרצה / הקבוצה

```
In [ ]: # התקנת ספריות NLP
!pip install spacy gensim pyLDAvis -q
# (אם קיים) spaCy הורדת מודל עברית ל
# !python -m spacy download he_core_news_sm # מודל עברית - בדוק זמינות
!pip install sentence-transformers scikit-learn -q
print("✅ הותקנו ספריות NLP")
print("    • spacy           - NLP pipeline")
print("    • gensim           - Topic Modeling (LDA)")
print("    • pyLDAvis          - ויזואליזציה של נושאים")
print("    • sentence-transformers - Embeddings")
```

```
In [ ]: # יבוא ספריות
import requests
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib
import re
import time
from collections import Counter
from IPython.display import display, HTML
import os

# NLP
import gensim
from gensim import corpora, models
from gensim.models import LdaModel
import pyLDAvis
import pyLDAvis.gensim_models as gensimvis
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.decomposition import NMF, LatentDirichletAllocation as LDA_sklearn
from sklearn.metrics.pairwise import cosine_similarity

matplotlib.rcParams['axes.unicode_minus'] = False
print("✅ יבוא ספריות")
print(f"    Gensim {gensim.__version__}")
```

חלק ב: טוקניזציה ותיוג מורפולוגי

מהי טוקניזציה?

Tokenization = פיצול טקסט ליחידות משמעות (tokens).

לעברית, זה מורכב יותר מאנגלית:

- "לבית" → [ל+בית] = 2 מורפמות
- "שלנו" → [של+נו] = גוף שני רבים
- "בבית" → [ב+בית]

spaCy לעברית:

```
import spacy
nlp = spacy.load("he_core_news_sm")
doc = nlp("ירושלים היא בירת מדינת ישראל")
for token in doc:
    print(token.text, token.pos_, token.dep_)
```

חלקי דיבור (POS Tags):

תג	שם	דוגמה
NOUN	שם עצם	ירושלים, חפירה
VERB	פועל	מצא, חפר
ADJ	שם תואר	עתיק, דיגטלי
PROPN	שם פרטי	דוד, חמאס
NUM	מספר	שלושה, 42

```
In [ ]: # =====
# טוקניזציה בסיסית לעברית
# =====

def tokenize_hebrew(text):
    """
    טוקניזציה בסיסית לעברית - פיצול למילים עם ניקוי
    YAP או HebSpaCy-הערה: לניתוח מתקדם, מומלץ להשתמש ב
    """
    # HTML ו-URLs הסרת
    text = re.sub(r'http\S+', ' ', text)
    text = re.sub(r'<[^>]+>', ' ', text)
    # שמירת אותיות עבריות ורווחים
    text = re.sub(r'^\u05D0-\u05EA\s$', ' ', text)
    # פיצול ושמירת מילים  $\geq 2$  אותיות
    tokens = [w for w in text.split() if len(w) >= 2]
    return tokens

def pos_tag_simple(tokens):
    """
    (ML ללא מודל) פשוט לעברית POS תיוג
    מבוסס על רשימות ידועות
    """
    # רשימות לדוגמה (מינימליות)
    proper_nouns = {
        'ירושלים', 'תל', 'אביב', 'חיפה', 'באר', 'שבע', 'ישראל',
        'יהודה', 'שומרון', 'גליל', 'נגב', 'סיני', 'ירדן',
        'מגידו', 'חצור', 'לכיש', 'באר', 'שובה', 'ערד', 'תמנע'
    }

    verbs_patterns = ['מצא', 'גילה', 'נבנה', 'נהרס', 'שלט', 'כבש', 'ניצח']

    tagged = []
    for token in tokens:
```

```

if token in proper_nouns:
    tagged.append((token, 'PROPN'))
elif any(token.startswith(v) or token.endswith(v) for v in verbs_pat):
    tagged.append((token, 'VERB'))
elif len(token) <= 3:
    tagged.append((token, 'FUNC')) # מילת תפקיד
else:
    tagged.append((token, 'NOUN')) # ברירת מחדל

return tagged

# — דוגמה —
example_texts = [
    "חפירות ארכיאולוגיות בירושלים חשפו שכבות מהתקופה הברונזה",
    "תל מגידו מציג שכבות מהמאה העשירית לפנה ספירה",
    "הגניזה הקהירית מכילה מסמכים מהמאה התשיעית עד המאה העשרים"
]

print("🔍 דוגמאות טוקניזציה:")
print("=" * 55)
for text in example_texts:
    tokens = tokenize_hebrew(text)
    tagged = pos_tag_simple(tokens)

    print(f"\n📄 טקסט: {text[:50]}...")
    print(f"   טוקנים: {tokens[:8]}")
    print(f"   תיוג: {[ (t, pos) for t, pos in tagged[:5]]}")

```

חלק ג: זיהוי ישויות בשם (NER)

מהו NER?

Named Entity Recognition = זיהוי ישויות בשם בטקסט.

סוגי ישויות:

סוג	תיאור	דוגמה
PER	אדם	"דוד המלך", "רבי עקיבא"
LOC	מקום	"ירושלים", "כינרת"
ORG	ארגון	"האוניברסיטה העברית"
DATE	תאריך/תקופה	"המאה ה-10 לפנה"ס"
EVENT	אירוע	"חורבן הבית"

חשיבות NER לאסטורים:

- ניתוח רשתות חברתיות – מי מזכיר את מי?
- מפה מקומות – בניית GeoHuman

• ציר זמן – מתי מתרחשים אירועים?

• ביוגרפיה – מי היה מה ומתי?

```
In [ ]: # =====
# לעברית (NER) זיהוי ישויות בשם
# =====

# (Gazetteer approach) רשימות ישויות ידועות
GAZETTEER = {
    'PER': { # אנשים
        'דוד', 'שלמה', 'שאול', 'אברהם', 'משה', 'יהושע',
        'בן גוריון', 'הרצל', 'רמב"ם', 'רש"י', 'ביאליק',
        'רבי עקיבא', 'יוסף פלביוס', 'בר כוכבא'
    },
    'LOC': { # מקומות
        'ירושלים', 'תל אביב', 'חיפה', 'באר שבע', 'יריחו',
        'מגידו', 'חצור', 'לכיש', 'ערד', 'תמנע', 'מצדה',
        'כינרת', 'ים המלח', 'ירדן', 'נגב', 'גליל', 'יהודה',
        'שומרון', 'עמק יזרעאל', 'שפלה', 'חרמון', 'כרמל'
    },
    'ORG': { # ארגונים
        'האוניברסיטה העברית', 'רשות העתיקות', 'מוזיאון ישראל',
        'האקדמיה ללשון עברית', 'הכנסת', 'מדינת ישראל',
        'הסוכנות היהודית', 'הבונד', 'הפועל'
    },
    'PERIOD': { # תקופות
        'הברונזה', 'הברזל', 'הביזנטית', 'הצלבנית', 'העות\מאנית',
        'הכנענית', 'הפרסית', 'ההלניסטית', 'הרומית', 'המנדטורית',
        'ימי הביניים', 'העת החדשה', 'העת העתיקה'
    }
}

def find_entities(text, gazetteer=None):
    """
    זיהוי ישויות בשם בטקסט עברי.

    (מבוסס רשימות ידועות) Gazetteer-based שיטה.

    עם מודל עברית spaCy, YAP, HebSpaCy: לזיהוי מתקדם יותר

    פרמטרים:
        text : הטקסט לניתוח
        gazetteer : מילון ישויות {סוג: set(ישויות)}

    מחזיר: רשימת (ישות, סוג, מיקום)
    """
    if gazetteer is None:
        gazetteer = GAZETTEER

    entities = []

    for entity_type, entity_list in gazetteer.items():
        for entity in entity_list:
            # חיפוש כל הופעות
```

```

        for match in re.finditer(re.escape(entity), text):
            entities.append({
                'ישות': entity,
                'סוג': entity_type,
                'התחלה': match.start(),
                'סיום': match.end()
            })

# מיון לפי מיקום
entities.sort(key=lambda x: x['התחלה'])
return entities

def highlight_entities(text, entities):
    """
    HTML פורמט הדגשת ישויות בטקסט
    """
    COLORS = {
        'PER': '#FFD700', # זהב - אנשים
        'LOC': '#90EE90', # ירוק - מקומות
        'ORG': '#87CEEB', # כחול - ארגונים
        'PERIOD': '#FFB347', # כתום - תקופות
    }

    result = text
    # מהסוף להתחלה כדי לא לשבור אינדקסים
    for entity in sorted(entities, key=lambda x: x['התחלה'], reverse=True):
        color = COLORS.get(entity['סוג'], '#DDDDDD')
        start, end = entity['התחלה'], entity['סיום']
        original = text[start:end]
        replacement = f'<mark style="background-color:{color}">{original}</su
        result = result[:start] + replacement + result[end:]

    return result

# — דוגמאות —
demo_texts = [
    "דוד המלך בנה את ירושלים לבירת ממלכת ישראל בתקופת הברזל",
    "חפירות ברשות העתיקות בתל מגידו חשפו ממצאים מהתקופה הכנענית",
    "הרמב"ם נולד בקורדובה ועבר לקהיר שם כתב את משנה תורה"
]

print("📄 זיהוי ישויות בשם (NER):")
print("=" * 65)

for text in demo_texts:
    entities = find_entities(text)
    print(f"\n📝 {text}")
    print(f"ישויות שזוהו: {len(entities)}")
    for e in entities:
        print(f"    [{e['סוג']}:6] {e['ישות']}"

# HTML הצגה ב
highlighted = highlight_entities(text, entities)
print()

```

```
print("📌 מקרא:")
print(" 🟡 PER = אדם | 🟢 LOC = מקום | 🔵 ORG = ארגון | 🟠 PERIOD = זמן")
```

חלק ד: מידול נושאים (Topic Modeling)

מהו Topic Modeling?

Topic Modeling = גילוי נושאים נסתרים בקורפוס גדול, ללא פיקוח (Unsupervised Learning).

האלגוריתם המרכזי: LDA

LDA (Latent Dirichlet Allocation) הנחה:

- כל מסמך = תערובת של נושאים
- כל נושא = התפלגות מעל מילים
- מילים = הנראות; נושאים = הנסתר

דוגמה:

"מסמך": "חפירה בתל מגידו חשפה חרסים מהתקופה הכנענית
 נושא 1 (ארכיאולוגיה): חפירה=0.15, תל=0.12, חרסים=0.11
 נושא 2 (תקופות): כנענית=0.18, ברזל=0.14, ברונזה=0.13

שימושים במדעי הרוח:

- גילוי נושאים מרכזיים בקורפוס עיתונות היסטורית
- מעקב אחר שינויים בשיח לאורך זמן
- זיהוי אשכולות של טקסטים דומים

Blei, D.M., Ng, A.Y., Jordan, M.I. (2003). *Latent Dirichlet Allocation*. JMLR 📖

```
In [ ]: # =====
# הכנת הקורפוס למידול נושאים
# =====

# טקסטים לדוגמה (בפרויקט אמיתי - נשתמש בקורפוס מוויקיפדיה)
SAMPLE_CORPUS = [
    # ארכיאולוגיה
    "חפירות בתל מגידו חשפו שכבות מהתקופה הכנענית והברונזה",
    "ממצאים ארכיאולוגיים ברמלה כוללים קרמיקה מהתקופה האסלאמית",
    "חפירה בירושלים גילתה חותם מלכותי מהתקופה הברזל",
    "אתר ארכיאולוגי בתל חצור מציג שרידי חומה מהמאה העשירית",
    "ממצאי ברונזה מאתר לכיש מעידים על תרבות עשירה",
    "חפירות בתל ערד חשפו מקדש ישראלי מהתקופה המלוכה",
    "קרמיקה פלשתית נמצאה באתרים רבים בשפלה יהודה",

    # היסטוריה
    "דוד המלך ייסד את ירושלים לבירת ממלכת ישראל",
```



```

"חורבן בית המקדש הראשון בשנת תקפ לפנה ספירה",
"הגלות הבבלית השפיעה עמוקות על התרבות היהודית",
"תקופת בית שני כוללת השפעה פרסית הלניסטית ורומית",
"המרד הגדול ברומאים הסתיים בחורבן הבית השני",
"תקופת הגאונים בבבל ייצרה ספרות הלכתית עשירה",
"היגירה לספרד הולכה לתרחיש של ימי הביניים",

# מדעי הרוח הדיגיטליים
"כלים דיגיטליים לניתוח טקסטים היסטוריים עבריים",
"GIS ויזואליזציה של נתונים ארכיאולוגיים באמצעות",
"מסדי נתונים לתיעוד ממצאים ארכיאולוגיים",
"עיבוד שפה טבעית לניתוח הגניזה הקהירית",
"מפות דיגיטליות לחקר ישובים היסטוריים",
"בינה מלאכותית לזיהוי אתרים ארכיאולוגיים בצילומי אוויר",
]

# מילות עצירה עבריות
STOPWORDS = set([
    'של', 'עם', 'את', 'אל', 'על', 'כי', 'כן', 'לא', 'הוא', 'היא',
    'הם', 'הן', 'זה', 'זו', 'כל', 'יש', 'אין', 'רק', 'גם',
    'אבל', 'אם', 'כאשר', 'בין', 'מהתקופה', 'מהמאה', 'לפנה',
    'ב', 'ו', 'ה', 'ל', 'מ', 'כ', 'ש'
])

def preprocess_for_lda(texts, stopwords=None):
    """
    עיבוד טקסטים למידול LDA.

    פילטור קצרות → stop words שלבים: ניקוי → טוקניזציה → הסרת
    """
    if stopwords is None:
        stopwords = STOPWORDS

    processed = []
    for text in texts:
        # ניקוי
        text = re.sub(r'^\u05D0-\u05EA\s', ' ', text)
        # טוקניזציה וסינון
        tokens = [
            w for w in text.split()
            if len(w) >= 3 and w not in stopwords
        ]
        processed.append(tokens)

    return processed

# עיבוד הקורפוס
processed_corpus = preprocess_for_lda(SAMPLE_CORPUS)

# Bag of Words-יצירת מילון ו
dictionary = corpora.Dictionary(processed_corpus)
bow_corpus = [dictionary.doc2bow(doc) for doc in processed_corpus]

print(f"✅ הקורפוס עובד")

```

```

print(f" מסמכים: {len(processed_corpus)}")
print(f" מילים במילון: {len(dictionary)}")
print(f"\n 🔍 מסמך ראשון - דוגמה:")
print(f" מילים: {processed_corpus[0]}")
print(f" BoW: {bow_corpus[0][:5]}")

```

In []:

```

# =====
# אימון מודל LDA
# =====

print("👉 מאמן מודל LDA...")

# הפרמטר החשוב ביותר: num_topics
# Coherence Score בפרויקט אמיתי - נרצה לבדוק ערכים שונים ולמדוד
NUM_TOPICS = 4 # ניסיון ראשוני

lda_model = LdaModel(
    corpus=bow_corpus,
    id2word=dictionary,
    num_topics=NUM_TOPICS,
    random_state=42, # לשחזוריות
    update_every=1,
    chunksize=10,
    passes=20, # כמה פעמים לעבור על הקורפוס
    alpha='auto', # אלפא = מידת ערוב הנושאים
    per_word_topics=True
)

print(f"✅ אומן LDA מודל")
print(f" נושאים: {NUM_TOPICS}")
print()

# הצגת הנושאים
print("📊 הנושאים שהתגלו")
print("=" * 60)

for topic_id, words in lda_model.print_topics(num_topics=NUM_TOPICS, num_words=10):
    print(f"\n נושא {topic_id + 1}:")
    # פירסור המילים
    word_weights = []
    for item in words.split(' + '):
        weight, word = item.split('*')
        word = word.replace('\n', '')
        word_weights.append((word, float(weight)))

    for word, weight in word_weights:
        bar = '█' * int(weight * 200)
        print(f" {word:<15} {weight:.4f} |{bar}")

```

In []:

```

# =====
# ויזואליזציה של הנושאים
# =====

# — גרף: מילים מובילות לכל נושא —
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

```

```

axes = axes.flatten()

colors_list = ['#e74c3c', '#2980b9', '#27ae60', '#f39c12']

for topic_id in range(NUM_TOPICS):
    topic = lda_model.show_topic(topic_id, topn=10)
    words, weights = zip(*topic)

    ax = axes[topic_id]
    bars = ax.barh(range(len(words)), weights,
                   color=colors_list[topic_id], alpha=0.8)
    ax.set_yticks(range(len(words)))
    ax.set_yticklabels(words, fontsize=11)
    ax.invert_yaxis()
    ax.set_title(f'נושא {topic_id + 1}', fontsize=13, fontweight='bold',
                 color=colors_list[topic_id])
    ax.set_xlabel('משקל', fontsize=10)

plt.suptitle('(מילים מובילות לכל נושא) LDA - מידול נושאים',
             fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig('01_lda_topics.png', dpi=150, bbox_inches='tight')
plt.show()
print("📁 01 הגרף נשמר: lda_topics.png")

# — pyLDAvis ויזואליזציה אינטראקטיבית עם —
print("\n🌐 יוצר ויזואליזציה אינטראקטיבית")
try:
    vis_data = gensimvis.prepare(lda_model, bow_corpus, dictionary)
    pyLDAvis.save_html(vis_data, '02_lda_interactive.html')
    print("✅ 02 ויזואליזציה נשמרה: lda_interactive.html")
    print("🔗 פתחו את הקובץ בדפדפן לחקירה אינטראקטיבית")
except Exception as e:
    print(f"⚠️ pyLDAvis ויזואליזציה: {e}")
    print("🛑 ניתן להמשיך - הגרפים הסטטיים זמינים")

```

חלק ה: Word Embeddings – מילים כוקטורים

מהו Word Embedding?

Word Embedding = ייצוג מילים כוקטורים במרחב רב-ממדי.

הרעיון המרכזי:

"מילים המופיעות בהקשרים דומים = ייצוגים דומים במרחב הוקטורי"

דוגמאות קלאסיות:

- מלך – גבר + אישה ≈ מלכה
- ירושלים – ישראל + צרפת ≈ פריז

מודלים מפורסמים:

מודל	מפתח	שנה	מאפיין
Word2Vec	Google	2013	מהיר, פשוט
GloVe	Stanford	2014	מבוסס קו-אוקורנס גלובלי
FastText	Meta	2016	טוב למורפולוגיה (= עברית!)
BERT	Google	2018	הקשרי, שינה את ה-NLP

FastText לעברית:

FastText מיוחד לשפות מורפולוגיות כמו עברית כי הוא עובד ברמת ה**תת-מילה** (subword).

```
In [ ]: # =====
# (גרסה פשוטה) Word Embeddings הדגמת
# =====
# כייצוג וקטורי בסיסי TF-IDF matrix - בגרסה פשוטה - נשתמש ב
# FastText, BERT-Hebrew, AlephBERT: לייצוגים מתקדמים

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA

print("📊 (TF-IDF) מחשב ייצוגים וקטוריים")

# TF-IDF ייצוג
tfidf = TfidfVectorizer(analyzer=lambda x: x) # קורא רשימת מילים
tfidf_matrix = tfidf.fit_transform(processed_corpus)

print(f"ממדי המטריצה: {tfidf_matrix.shape}")
print(f"({tfidf_matrix.shape[0]} מסמכים × {tfidf_matrix.shape[1]} מילים)")

# — PCA: הקטנת ממדים לויזואליזציה —
pca = PCA(n_components=2, random_state=42)
doc_2d = pca.fit_transform(tfidf_matrix.toarray())

# — D גרף: מסמכים במרחב 2 —
colors_map = ['#e74c3c'] * 7 + ['#2980b9'] * 7 + ['#27ae60'] * 6
labels = ['ארכיאולוגיה'] * 7 * ['היסטוריה'] + 7 + ['DH'] * 6

fig, ax = plt.subplots(figsize=(10, 7))
for cat, color in [('#27ae60', 'היסטוריה'), ('#2980b9', 'היסטוריה'), ('#e74c3c', 'ארכיאולוגיה')]:
    mask = [l == cat for l in labels]
    xs = [doc_2d[i][0] for i in range(len(mask)) if mask[i]]
    ys = [doc_2d[i][1] for i in range(len(mask)) if mask[i]]
    ax.scatter(xs, ys, c=color, label=cat, s=120, alpha=0.8, edgecolors='white')

# הוספת מספרי מסמכים
for i, (x, y) in enumerate(doc_2d):
    ax.annotate(str(i+1), (x, y), textcoords='offset points',
                xytext=(6, 4), fontsize=8, color='gray')
```

```

ax.set_xlabel('PCA 1', fontsize=12)
ax.set_ylabel('PCA 2', fontsize=12)
ax.set_title('מסמכים קרובים = תוכן דומה (PCA)\ מסמכים במרחב וקטורי',
             fontsize=13, fontweight='bold')
ax.legend(fontsize=11)
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('03_document_space.png', dpi=150, bbox_inches='tight')
plt.show()
print("📁 03 הגרף נשמר: document_space.png")

# — דמיון בין מסמכים —
print("\n🔍 דמיון בין מסמכים (Cosine Similarity):")
sim_matrix = cosine_similarity(tfidf_matrix)
print(f"מסמך 1 ↔ מסמך 2 (ארכיאולוגיה) {sim_matrix[0][1]:.3f}")
print(f"מסמך 1 ↔ מסמך 8 (היסטוריה) {sim_matrix[0][7]:.3f}")
print(f"מסמך 1 ↔ מסמך 15 (DH): {sim_matrix[0][14]:.3f}")

```

חלק ו: יישומים במדעי הרוח

דוגמאות מחקריות:

1. ניתוח הגניזה הקהירית
מאגר Friedberg Genizah Project – שימוש ב-NLP לניתוח 330,000 קטעים.
2. ניתוח עיתונות היסטורית
עיתונים עבריים מ-1870 עד 1948 – topic modeling לזיהוי שינויי שית.
3. זיהוי דמויות היסטוריות
NER + co-occurrence analysis = מי הכיר את מי בתקופה התלמודית?
4. ניתוח ספרותי
השוואת קורפוסים: שפת ביאליק לעומת עגנון – מה מייחד כל אחד?

כלים מתקדמים לעברית:

- **AlephBERT**: מודל BERT מאומן על עברית (AI21 Labs)
- **HebSpaCy**: spaCy עם תמיכה בעברית
- **MILA**: מאגר NLP לעברית (Technion)
- **DictaBERT**: BERT עם מורפולוגיה עברית

```

In [ ]: # =====
# גרסה פשוטה – (Sentiment Analysis) ניתוח רגש
# =====

# מילוני רגש עבריים בסיסיים
POSITIVE_WORDS = {
    'טוב', 'מצוין', 'נפלא', 'יפה', 'חשוב', 'מרשים', 'ייחודי',

```

```

'נדיר', 'מדהים', 'משמעותי', 'חיוני', 'ערך', 'עשיר',
'מתקדם', 'חדשני', 'פורה', 'הצלחה', 'ניצחון', 'שגשוג',
'שלום', 'ברית', 'קדוש', 'חכמה', 'גדול', 'כביר'
}

NEGATIVE_WORDS = {
    'רע', 'גרוע', 'נורא', 'חרב', 'הרוס', 'נפל', 'מסכן',
    'מלחמה', 'כיבוש', 'גלות', 'גירוש', 'רצח', 'הרג',
    'חורבן', 'כישלון', 'אסון', 'שבוי', 'עצב', 'בכי',
    'עוני', 'רעב', 'מחלה', 'אויב', 'פגע', 'הגלה'
}

def analyze_sentiment(text, pos_words=None, neg_words=None):
    """
    ניתוח רגש בסיסי לטקסט עברי.

    עם ציון רגש ומילים שהתגלו ממשזר: dict
    """
    if pos_words is None:
        pos_words = POSITIVE_WORDS
    if neg_words is None:
        neg_words = NEGATIVE_WORDS

    # ניקוי וטוקניזציה
    words = re.sub(r'^\u05D0-\u05EA\s', '', text).split()

    found_positive = [w for w in words if w in pos_words]
    found_negative = [w for w in words if w in neg_words]

    pos_score = len(found_positive)
    neg_score = len(found_negative)

    if pos_score > neg_score:
        sentiment = 'חיובי 😊'
    elif neg_score > pos_score:
        sentiment = 'שלילי 😞'
    else:
        sentiment = 'ניטרלי 😐'

    return {
        'רגש': sentiment,
        'ציון_חיובי': pos_score,
        'ציון_שלילי': neg_score,
        'מילים_חיוביות': found_positive,
        'מילים_שליליות': found_negative
    }

# — דוגמאות —
print("😊: ניתוח רגש לטקסטים היסטוריים")
print("=" * 65)

sentiment_examples = [
    "ירושלים שגשגה בתקופת שלמה המלך, המלך הגדול שבנה את המקדש הנפלא",
    "חורבן הבית הוביל לגלות ורצח והרס נורא של עם ישראל",

```

```

"הממצאים הארכיאולוגיים מספקים מידע חשוב על העבר ההיסטורי",
"המלחמה והכיבוש הרומי גרמו לאסון ולמחלה ולעוני בארץ"
]

for text in sentiment_examples:
    result = analyze_sentiment(text)
    print(f"\n📄 {text[:55]}...")
    print(f"    רגש: {result['רגש']}")
    print(f"    חיובי ({result['חיוב_חיובי']}): {result['מילים_חיוביות']}")
    print(f"    שלילי ({result['שליל_שלילי']}): {result['מילים_שליליות']}")

print("\n⚠️ הערה: ניתוח רגש בסיסי בלבד - לניתוח מדויק")
print("    pip install transformers # AlephBERT (בעברית)")

```

```

In [ ]: # =====
# שמירת הנתונים
# =====

import pandas as pd

print("📊 שומר נתונים...")

# שמירת הקורפוס המעובד
corpus_data = []
for i, (original, processed) in enumerate(zip(SAMPLE_CORPUS, processed_corpus)):
    topic_dist = lda_model.get_document_topics(bow_corpus[i])
    top_topic = max(topic_dist, key=lambda x: x[1])[0] if topic_dist else -1
    corpus_data.append({
        'מזהה': i + 1,
        'טקסט': original,
        'מילים': ' '.join(processed),
        'נושא_ראשי': top_topic + 1
    })

df_corpus = pd.DataFrame(corpus_data)
df_corpus.to_csv('nlp_corpus.csv', index=False, encoding='utf-8-sig')
print("✅ nlp_corpus.csv")

# שמירת נושאי LDA
topics_data = []
for topic_id in range(NUM_TOPICS):
    for word, weight in lda_model.show_topic(topic_id, topn=10):
        topics_data.append({
            'נושא': topic_id + 1,
            'מילה': word,
            'משקל': round(weight, 4)
        })

pd.DataFrame(topics_data).to_csv('lda_topics.csv', index=False, encoding='utf-8-sig')
print("✅ lda_topics.csv")

print("\n📄 מחברת 4 הושלמה!")
print("\n📖 הכלים המתקדמים לעברית")
print("    → AlephBERT: https://huggingface.co/onlplab/alephbert-base")
print("    → DictaBERT: https://huggingface.co/dicta-il")
print("    → HebSpaCy: https://github.com/explosion/spacy-models")

```

תרגילים

תרגיל 1 – בסיסי

הריצו LDA עם מספר נושאים שונה (2, 4, 6, 8) ובדקו:
אילו נושאים מתאימים יותר לנתונים שלנו?

תרגיל 2 – בינוני

הוסיפו ישויות חדשות ל-Gazetteer (מקומות, דמויות, ארגונים שאתם מכירים מהתחום).
הריצו NER על טקסטים מהקורפוס שנאסף במחברת 1.

תרגיל 3 – מתקדם

השתמשו ב-AlephBERT לניתוח רגש מתקדם:

```
from transformers import pipeline
# nlp_he = pipeline("text-classification",
# model="avichr/heBERT_sentiment_analysis")
```

משאבים

- BERT – AlephBERT לעברית
- Gensim LDA Tutorial
- pyLDavis – ויזואליזציה של LDA
- Blei et al. (2003). *Latent Dirichlet Allocation*. JMLR
- Programming Historian: NLP

משימת הבית

מה להגיש:

1. הריצו LDA על קורפוס אחר – שנו את רשימת הדפים ב-WIKI_PAGES לנושא שמעניין אתכם (לפחות 15 דפים)
2. תארו נושא אחד – בחרו נושא אחד מהתוצאות וכתבו פסקה: על מה לדעתכם הנושא הזה? אילו מילות מפתח מרמזות על כך?
3. ההרור ביקורתי – ב-2-3 משפטים: מה המגבלות של שיטת LDA? מה היא לא יכולה לאתר?

כיצד להגיש:

1. File → Save a copy in Drive – ודאו שהמחברת שמורה

2. `.ipynb` Download → Download → File → הורידו את הקובץ

3. העלו ל-Moodle

🤖 עצה: אם נתקעתם – שאלו את Gemini!

פתחו טאב חדש ב-[Google AI Studio](#) ושאלו:

Python-ומידול נושאים ב NLP אני לומד/ת.
"נתקלתי בשגיאה הזו: "[הדביקו את השגיאה כאן]
?מה המשמעות ואיך אפשר לפתור